# Live Traffic Monitor — Problem Requirements

## Overview

Build a pure Python live traffic monitoring system.

**Main function:** `create_traffic_monitor(service_name, max_rps, allowed_endpoints)`
**Returns:** A single `record(user_id, endpoint, response_time, status_code)` function

---

## Layer 1 — Validation

| Input | Rule | Action |
|---|---|---|
| `user_id` | Cannot be None | raise `TypeError` |
| `user_id` | Cannot be empty string | raise `ValueError` |
| `user_id` | Must be a string | raise `TypeError` |
| `response_time` | If integer → convert to float silently | convert |
| `response_time` | If negative or zero → reject | raise `ValueError` |
| `status_code` | Must be int between 100–599 | raise `ValueError` if not |

---

## Layer 2 — Rate Limiting

- Each user gets max `max_rps` requests per second
- Store request timestamps per user in a dict
- Clean old timestamps outside 1-second window on every call
- If user exceeds limit → return blocked response, do not raise error

---

## Layer 3 — Analytics (all via nonlocal)

Track all of these across all calls:

- Total requests
- Total blocked
- Total server errors (5xx)
- Average response time (non-blocked only)
- Unique users (most memory-efficient structure)
- Suspicious requests (endpoint not in whitelist)
- Every 50 requests → print stats summary to console

---

## Layer 4 — Status Classification (use match/case)

| Status code range | Classification |
|---|---|
| 200–299 | `"success"` |
| 400–499 | `"client_error"` |
| 500–599 | `"server_error"` |

---

## Layer 5 — Security

- Check if endpoint is in `allowed_endpoints` set
- If not → classify as `"suspicious"`, track separately, return suspicious response
- Use set membership check ( `in` operator)

---

## Layer 6 — Slow Requests Generator

- Attach `get_slow_requests(threshold)` as attribute on the `record` function
- It is a generator that yields all recorded requests where `response_time > threshold`
- Each item yielded: `(user_id, endpoint, response_time, status_code)`
- Store only last 500 requests — use slicing to trim

## Layer 7 — Identity + Mutability

- Every response must contain a fresh `stats` dict — never return the same object twice
- Two consecutive calls must never have `r1["stats"] is r2["stats"]` == True

---

## Return Values

**Normal allowed request:**

```
{
    "status": "allowed",
    "classification": "success",
    "user_id": "user_123",
    "endpoint": "/api/pay",
    "response_time": 0.23,
    "stats": {
        "total_requests": 5,
        "total_blocked": 0,
        "total_server_errors": 0,
        "avg_response_time": 0.31,
        "unique_users": 2,
        "suspicious_requests": 0
    }
}
```

**Blocked request:**

```
{
    "status": "blocked",
    "user_id": "user_123",
    "reason": "rate limit exceeded"
}
```

**Suspicious endpoint:**

```
{
    "status": "suspicious",
    "endpoint": "/admin/hack",
    "reason": "endpoint not whitelisted"
}
```


---

## Test Suite — All Must Pass

```
monitor = create_traffic_monitor(
    service_name="payment-service",
    max_rps=3,
    allowed_endpoints={"/api/pay", "/api/refund", "/api/status"}
)

record = monitor

# Test 1 — normal request
r1 = record("user_1", "/api/pay", 0.23, 200)
assert r1["status"] == "allowed"
assert r1["classification"] == "success"

# Test 2 — server error classification
r2 = record("user_1", "/api/pay", 0.45, 500)
assert r2["classification"] == "server_error"

# Test 3 — integer response time silent conversion
r3 = record("user_2", "/api/refund", 1, 200)
assert isinstance(r3["response_time"], float)

# Test 4 — rate limiting
for _ in range(3):
    record("user_3", "/api/status", 0.1, 200)
r_blocked = record("user_3", "/api/status", 0.1, 200)
assert r_blocked["status"] == "blocked"

# Test 5 — suspicious endpoint
r5 = record("user_1", "/admin/hack", 0.1, 200)
assert r5["status"] == "suspicious"

# Test 6 — ValueError for bad user_id
try:
    record("", "/api/pay", 0.1, 200)
    assert False, "should have raised"
except ValueError:
    pass
```

```
# Test 7 — stats dict is never the same object
r7a = record("user_1", "/api/pay", 0.3, 200)
r7b = record("user_2", "/api/pay", 0.3, 200)
assert r7a["stats"] is not r7b["stats"]

# Test 8 — slow requests generator
slow = list(record.get_slow_requests(0.4))
assert all(rt > 0.4 for _, _, rt, _ in slow)

print("All tests passed.")
```

---

## Build Order (recommended)

```
Step 1 — outer function skeleton + nonlocal variables
Step 2 — validation layer
Step 3 — rate limiting layer
Step 4 — match/case classification
Step 5 — analytics updates
Step 6 — build response dict (fresh every time)
Step 7 — get_slow_requests generator
Step 8 — run test suite
```